



Distributed Systems

Fault Tolerance & Recovery

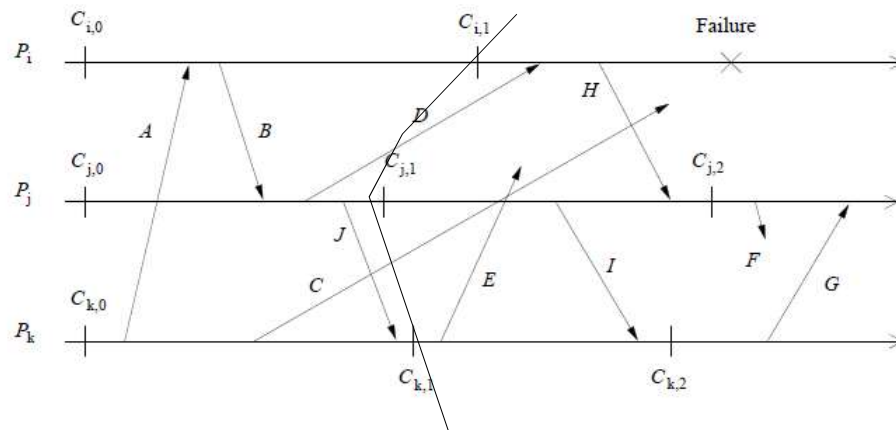


Recap



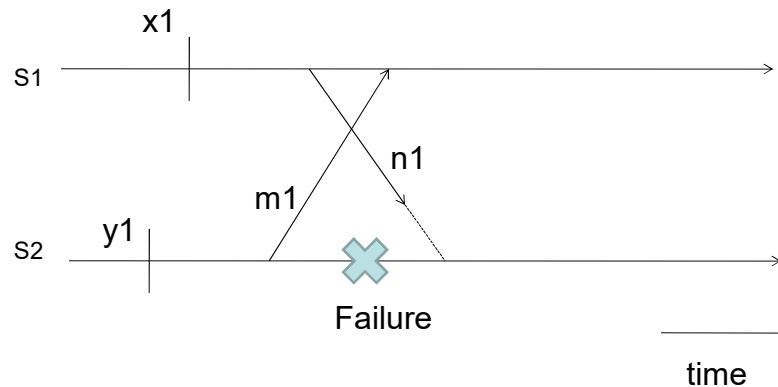
- 3PC
- Concurrency set / sender set
- Failure / timeout transitions
- Local / global checkpoint
- Consistent / strongly consistent checkpoint
- Different types of messages

Issues in failure recovery

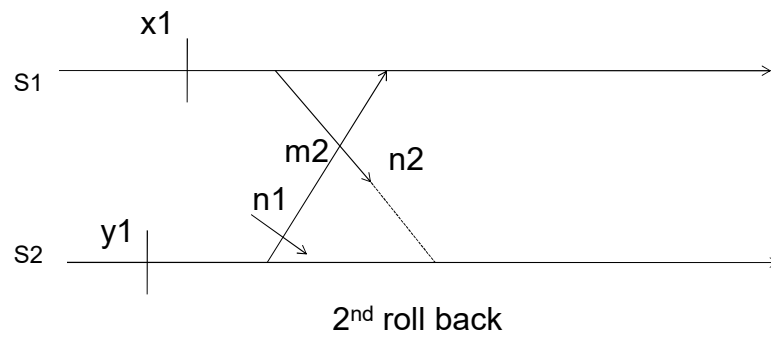


Normal, Vanished and other messages

Problem of Livelocks - It is a situation in which a single failure can cause an infinite number of rollbacks, preventing the system from making progress.

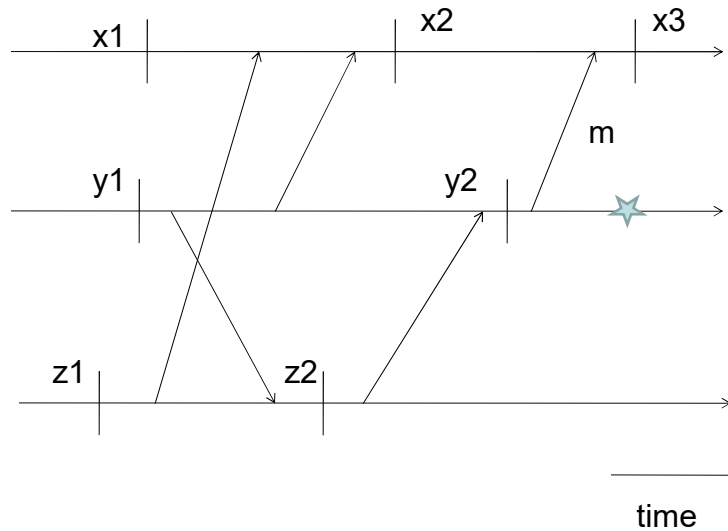


On recovery of process Y

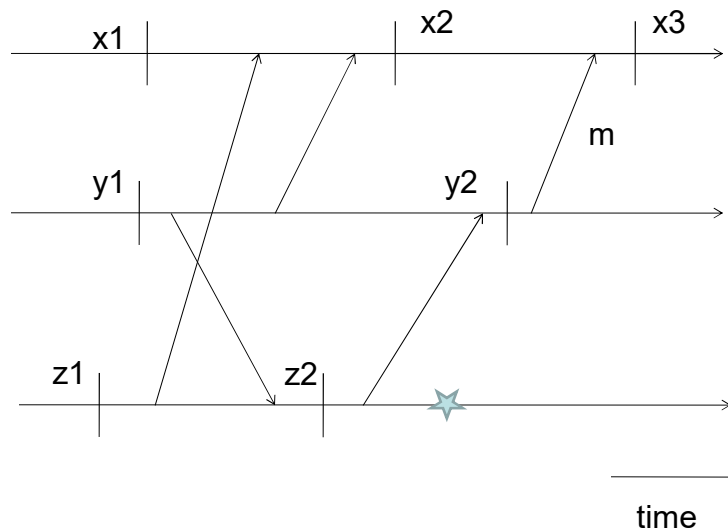


Domino effect

Domino effect: Where the rolling back one process causes one or more other processes to roll back.



Domino effect: Where the rolling back one process causes one or more other processes to roll back.



Performance measures for a checkpointing and recovery algorithm

- **during fault-free operation**
 - checkpointing time
 - space for storing checkpoints and messages (if needed)
- **in case of a fault**
 - recovery time (time to establish recovery line)
 - extent of rollback (how far in the past did we roll back? how much computation is lost?)
 - is **output commit problem** handled? (if an output was sent out before the fault, it should not be resent after restarting after the fault)

Some parameters that affect performance

- Checkpoint interval : time between two successive checkpoints
- Number of processes
- Communication pattern of the application
- Fault frequency
- Nature of stable storage

Classification of Checkpoint & Recovery Algorithms

- **Communication Induced**

- Since message send/receive is the fundamental cause of inconsistency in global checkpoint

Synchronize checkpointing with communication.

Ex. : take local checkpoint right after every send!

Last local checkpoint at each process is always consistent. But too costly

- Many variations are there, more efficient than the above

Classification of Checkpoint & Recovery Algorithms

- **Asynchronous/Uncoordinated**

- every process takes local checkpoint independently
- to recover from a fault in one process, all processes coordinate to find a consistent global checkpoint from their local checkpoints
- very low fault-free overhead, recovery overhead is high
- **Domino effect** possible (no consistent global checkpoint exist, so all processes have to restart from scratch)
- higher space requirements, as all local checkpoints need to be kept
- Good for systems where fault is rare and inter-process communication is not too high (less chance of domino effect)

- **Synchronous/Coordinated**

- all processes coordinate to take a consistent global checkpoint
- during recovery, every process just rolls back to its last local checkpoint independently
- low recovery overhead, but high checkpointing overhead
- no domino effect possible
- low space requirement, since only last checkpoint needs to be stored at each process

- Message logging

- Take coordinated or uncoordinated checkpoint, and then log (in stable storage) all messages received since the last checkpoint
- **On recovery, only the recovering process goes back to its last checkpoint**, and then replays messages from the log appropriately until it reaches the state right before the fault
- Only class that can handle output commit problem!

Some Checkpointing Algorithms

Asynchronous/Uncoordinated

- **Juang-Venkatesan's algorithm**

Synchronous/Coordinated

- Chandy-Lamport's global state collection algorithm can be modified to handle recovery from faults
- **Koo-Toueg's algorithm**

Quasi-synchronous

Juang and Venkatesan Algorithm

Assumptions:

- The communication channels are **reliable**, deliver the messages in **FIFO** order and have infinite buffers.
- The message transmission delay is arbitrary, but finite
- The processors directly connected to a processor via communication channels are called its neighbors.
- Two types of log storage: **volatile log and stable log**
- Periodically, a processor independently saves the contents of the volatile log in the stable storage and clears the volatile log - **local checkpoint**

The **underlying computation** is assumed to be event-driven

- A processor P waits until a message m is received, it processes the message m , changes its state from s to s' , and sends zero or more messages to some of its neighbors.
- Then the processor remains idle until the receipt of the next message.
- The new state s' and the contents of messages sent to its neighbors depends on state s and the contents of message m .

Juang and Venkatesan Algorithm

- All events (except the first or initial event) begin with the receipt of a message
- After an event, a processor records a triplet $\{s, m, \text{msgs_sent}\}$ in its volatile storage
 - s**: state of the processor before the event
 - m**: message (including the identity of the sender) whose arrival caused the event
 - msgs_sent**: set of messages sent by the during the event
- No synchronization with other processors

The Recovery Algorithm

Notations and data structure at P_i :

RCVD $_{i \leftarrow j}$ (CkPt $_i$): number of messages received by processor p_i from processor p_j , from the beginning of the computation till the checkpoint CkPt $_i$.

SENT $_{i \rightarrow j}$ (CkPt $_i$) represents the number of messages sent by processor p_i to processor p_j , from the beginning of the computation till the checkpoint CkPt $_i$.

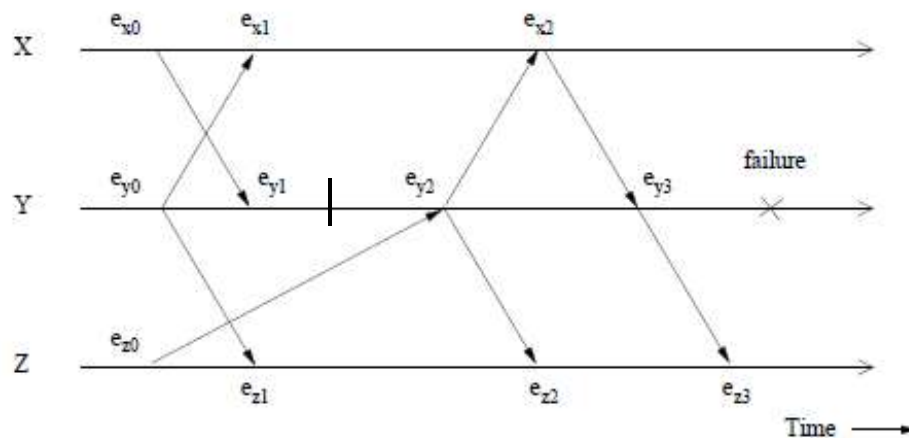
Main issue: Find a consistent set of checkpoints

- Each processor keeps track of both the number of messages it has sent to other processors as well as the number of messages it has received from others

Basic Idea:

- Whenever processor p_i rolls back, all other processors find out if any message sent by p_i has become an **orphan message**.
- In so, processor p_j also rolls back to a state where the number of messages received **agrees** with the number of messages sent
- Recovery may involve several iterations of roll backs by processors.

Example



The Algorithm (executed by processor P_i)

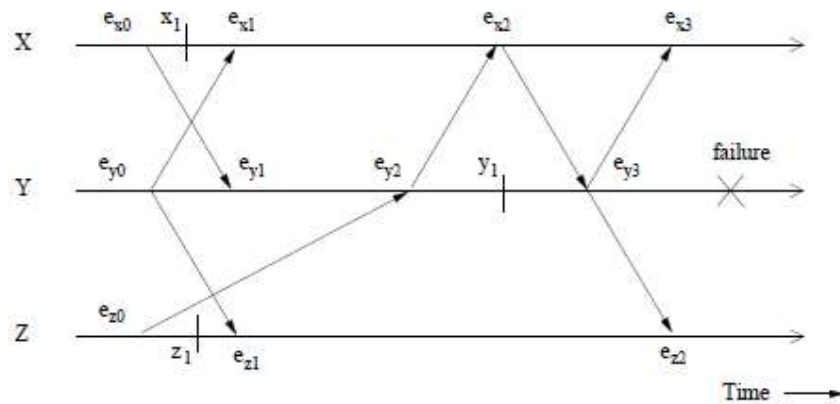
- The recovery algorithm at a processor is initiated when it restarts after a failure or when it learns of a failure at another processor.
- When a processor restarts after a failure, it broadcasts a ROLLBACK message that it had failed
- **STEP (a)**
 - if** processor p_i is recovering after a failure **then**
 - $CkPti :=$ latest event logged in the stable storage
 - else**
 - $CkPti :=$ latest event that took place in p_i {The latest event at p_i can be either in stable or in volatile storage.}
 - end if**

STEP (b)

```

for  $k = 1$  to  $N$  { $N$  is the number of processors} do
  for each neighboring processor  $p_j$  do
    compute  $SENT_{i \rightarrow j}(CkPti)$ 
    send a  $ROLLBACK(i, SENT_{i \rightarrow j}(CkPti))$  message to  $p_j$ 
  end for
  for every  $ROLLBACK(j, c)$  message received from a neighbor  $j$  do
    if  $RCVD_{i \leftarrow j}(CkPti) > c$  {Implies orphan messages}
    then
      find the latest event  $e$  such that  $RCVD_{i \leftarrow j}(e) \leq c$ 
      {Such an event  $e$  may be in the volatile or stable storage}
       $CkPti := e$ 
    end if
  end for
end for{for  $k$ }

```



Rollback to e_{x2}, e_{y2}, e_{z1}

- Procedure has $|N|$ iterations
- During the k th iteration ($k \neq 1$), a processor p_i does:
 - (i) based on the state $CkPt_i$ it was rolled back in the $(k-1)^{th}$ iteration, it computes $SENT_{i \rightarrow j}(CkPt_i)$ for each neighbor p_j and sends this value in a ROLLBACK message to neighbors and
 - (ii) p_i waits for and processes ROLLBACK messages received from its neighbors in k th iteration and determines a new recovery point $CkPt_i$ for p_i
- At the end of each iteration, at least one processor will rollback to its final recovery point (unless the current recovery points are already consistent)

Coordinated Checkpointing

- Processes coordinate their checkpointing activities so that all local checkpoints form a consistent global state
- Simplifies recovery
- No domino effect
- Each process to maintain only one checkpoint on the stable storage

Disadvantage

- Delay and Message overhead
- Large latency is involved in committing output, as a global checkpoint is needed before a message is sent to *Outside World Process*

Blocking Coordinated Checkpointing



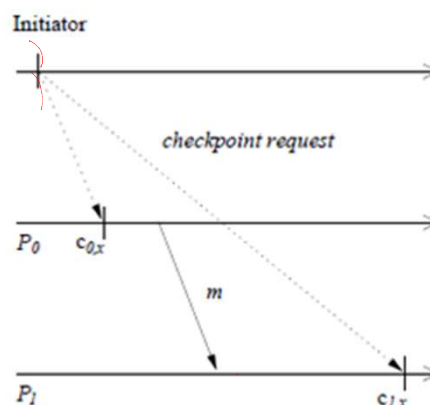
- Coordinator takes a checkpoint and broadcasts a request message to all, asking them to take a checkpoint.
- When a process receives this message, **it stops its execution, flushes all the communication channels, takes a tentative checkpoint, and sends back an ack.**
- After the coordinator receives acks from all processes, it broadcasts a commit message
- After receiving the commit message, a process removes the old permanent checkpoint **and atomically** makes the *tentative checkpoint permanent* and then resumes its execution

IIT ROORKEE

27

Non-blocking Checkpoint Coordination

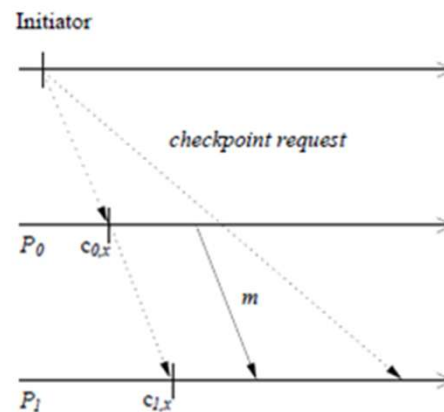
- No need to stop execution while taking checkpoint.
- How to prevent a process from receiving application messages that could make the checkpoint inconsistent.



FIFO channels:

- This problem can be avoided by preceding the first post-checkpoint message on each channel by a checkpoint request
- Forces each process to take a checkpoint before receiving the first post-checkpoint message

(Chandy and Lamport global snapshot algorithm)



Non-blocking Checkpoint Coordination

Non-FIFO channels:

- Piggyback marker on **every** post-checkpoint message
- When a process receives an application message with a marker, it treats it as if it has received a marker message, followed by the application message
- A checkpoint is triggered when the receiver's local checkpoint index is lower than the piggybacked checkpoint index.

Koo-Toueg Coordinated Checkpointing Algorithm

Assumptions:

- Communication channels are FIFO.
- **It is assumed that end-to-end protocols* exist to cope with message loss due to rollback recovery and communication failure.**
- Communication failures do not **partition** the network
- A single process invokes the algorithm to take permanent checkpoint
- No process fails during the execution of the algorithm.

Koo-Toueg Coordinated Checkpointing Algorithm

- The checkpoint algorithm takes two kinds of checkpoints on the stable storage:

Permanent and tentative

- **Permanent checkpoint:** A local checkpoint at a process that is a part of a consistent global checkpoint.
- **Tentative checkpoint:** A temporary checkpoint that is made a permanent checkpoint on the successful termination of the checkpoint algorithm.
- **In case of a failure, processes roll back only to their permanent checkpoints for recovery.**

First Phase

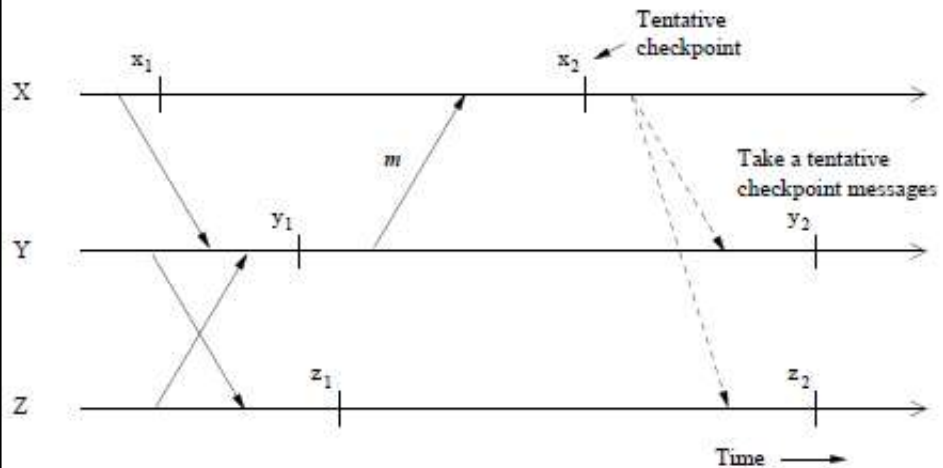
- An initiating process P_i takes a tentative checkpoint and requests all other processes to take tentative checkpoints. **(do not send messages now - blocking)**
- Each process informs P_i whether it succeeded in taking a tentative checkpoint or not.
- If P_i learns that all the processes have successfully taken tentative checkpoints, P_i decides that all tentative checkpoints should be made permanent;
- otherwise, P_i decides that all the tentative checkpoints should be discarded

Second Phase

- P_i informs all processes of the decision it reached at the end of the first phase.
- A process, on receiving the message from P_i acts accordingly (either all or none of the processes advance the checkpoint by taking permanent checkpoints)

After a process has taken a tentative checkpoint, it can not send messages related to the underlying computation until it is informed of P_i 's decision.

The protocol may cause a process to take a checkpoint even when it is not necessary for consistency.



Is it necessary for Y and Z to take a checkpoint??